



Apprenticeship Learning via Inverse Reinforcement Learning

Pieter Abbeel
Andrew Y. Ng

PABBEEL@CS.STANFORD.EDU
ANG@CS.STANFORD.EDU

Computer Science Department, Stanford University, Stanford, CA 94305, USA

Abstract

We consider learning in a Markov decision process where we are not explicitly given a reward function, but where instead we can observe an expert demonstrating the task that we want to learn to perform. This setting is useful in applications (such as the task of driving) where it may be difficult to write down an explicit reward function specifying exactly how different desiderata should be traded off. We think of the expert as trying to maximize a reward function that is expressible as a linear combination of known features, and give an algorithm for learning the task demonstrated by the expert. Our algorithm is based on using “inverse reinforcement learning” to try to recover the unknown reward function. We show that our algorithm terminates in a small number of iterations, and that even though we may never recover the expert’s reward function, the policy output by the algorithm will attain performance close to that of the expert, where here performance is measured with respect to the expert’s *unknown* reward function.

1. Introduction

Given a sequential decision making problem posed in the Markov decision process (MDP) formalism, a number of standard algorithms exist for finding an optimal or near-optimal policy. In the MDP setting, we typically assume that a reward function is given. Given a reward function and the MDPs state transition probabilities, the value function and optimal policy are exactly determined.

The MDP formalism is useful for many problems because it is often easier to specify the reward function than to directly specify the value function (and/or optimal policy). However, we believe that even the reward function is frequently difficult to specify manually. Consider, for example, the task of highway driving. When driving, we typically trade off many dif-

ferent desiderata, such as maintaining safe following distance, keeping away from the curb, staying far from any pedestrians, maintaining a reasonable speed, perhaps a slight preference for driving in the middle lane, not changing lanes too often, and so on To specify a reward function for the driving task, we would have to assign a set of weights stating exactly how we would like to trade off these different factors. Despite being able to drive competently, the authors do not believe they can confidently specify a specific reward function for the task of “driving well.”¹

In practice, this means that the reward function is often manually tweaked (cf. reward shaping, Ng et al., 1999) until the desired behavior is obtained. From conversations with engineers in industry and our own experience in applying reinforcement learning algorithms to several robots, we believe that, for many problems, the difficulty of manually specifying a reward function represents a significant barrier to the broader applicability of reinforcement learning and optimal control algorithms.

When teaching a young adult to drive, rather than telling them what the reward function is, it is much easier and more natural to demonstrate driving to them, and have them learn from the demonstration. The task of learning from an expert is called *apprenticeship learning* (also learning by watching, imitation learning, or learning from demonstration).

A number of approaches have been proposed for apprenticeship learning in various applications. Most of these methods try to directly mimic the demonstrator by applying a supervised learning algorithm to learn a direct mapping from the states to the actions. This literature is too wide to survey here, but some examples include Sammut et al. (1992); Kuniyoshi et al. (1994); Demiris & Hayes (1994); Amit & Mataric (2002); Pomerleau (1989). One notable exception is given in Atkeson & Schaal (1997). They considered the

¹We note that this is true even though the reward function may often be easy to state in English. For instance, the “true” reward function that we are trying to maximize when driving is, perhaps, our “personal happiness.” The practical problem however is how to model this (i.e., our happiness) explicitly as a function of the problems’ states, so that a reinforcement learning algorithm can be applied.

problem of having a robot arm follow a demonstrated trajectory, and used a reward function that quadratically penalizes deviation from the desired trajectory. Note however, that this method is applicable only to problems where the task is to mimic the expert’s trajectory. For highway driving, blindly following the expert’s trajectory would not work, because the pattern of traffic encountered is different each time.

Given that the entire field of reinforcement learning is founded on the presupposition that the reward function, rather than the policy or the value function, is the most succinct, robust, and transferable definition of the task, it seems natural to consider an approach to apprenticeship learning whereby the reward function is learned.²

The problem of deriving a reward function from observed behavior is referred to as inverse reinforcement learning (Ng & Russell, 2000). In this paper, we assume that the expert is trying (without necessarily succeeding) to optimize an unknown reward function that can be expressed as a linear combination of known “features.” Even though we cannot guarantee that our algorithms will correctly recover the expert’s true reward function, we show that our algorithm will nonetheless find a policy that performs as well as the expert, where performance is measured with respect to the expert’s *unknown* reward function.

2. Preliminaries

A (finite-state) Markov decision process (MDP) is a tuple (S, A, T, γ, D, R) , where S is a finite set of states; A is a set of actions; $T = \{P_{sa}\}$ is a set of state transition probabilities (here, P_{sa} is the state transition distribution upon taking action a in state s); $\gamma \in [0, 1)$ is a discount factor; D is the initial-state distribution, from which the start state s_0 is drawn; and $R : S \rightarrow A$ is the reward function, which we assume to be bounded in absolute value by 1. We let $\text{MDP} \setminus R$ denote an MDP without a reward function, i.e., a tuple of the form (S, A, T, γ, D) .

We assume that there is some vector of features $\phi : S \rightarrow [0, 1]^k$ over states, and that there is some “true” reward function $R^*(s) = w^* \cdot \phi(s)$, where $w^* \in \mathbb{R}^k$.³

²A related idea is also seen in the biomechanics and cognitive science, where researchers have pointed out that simple reward functions (usually ones constructed by hand) often suffice to explain complicated behavior (policies). Examples include the minimum jerk principle to explain limb movement in primates (Hogan, 1984), and the minimum torque-change model to explain trajectories in human multi-joint arm movement. (Uno et al., 1989) Related examples are also found in economics and some other literatures. (See the discussion in Ng & Russell, 2000.)

³The case of state-action rewards $R(s, a)$ offers no additional difficulties; using features of the form $\phi : S \times A \rightarrow [0, 1]^k$, and our algorithms still apply straightforwardly.

In order to ensure that the rewards are bounded by 1, we also assume $\|w^*\|_1 \leq 1$. In the driving domain, ϕ might be a vector of features indicating the different desiderata in driving that we would like to trade off, such as whether we have just collided with another car, whether we’re driving in the middle lane, and so on. The (unknown) vector w^* specifies the relative weighting between these desiderata.

A policy π is a mapping from states to probability distributions over actions. The value of a policy π is

$$E_{s_0 \sim D}[V^\pi(s_0)] = E[\sum_{t=0}^{\infty} \gamma^t R(s_t) | \pi] \quad (1)$$

$$= E[\sum_{t=0}^{\infty} \gamma^t w \cdot \phi(s_t) | \pi] \quad (2)$$

$$= w \cdot E[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) | \pi] \quad (3)$$

Here, the expectation is taken with respect to the random state sequence $s_0, s_1, \dots$ drawn by starting from a state $s_0 \sim D$, and picking actions according to π . We define the expected discounted accumulated feature value vector $\mu(\pi)$, or more succinctly the **feature expectations**, to be

$$\mu(\pi) = E[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) | \pi] \in \mathbb{R}^k. \quad (4)$$

Using this notation, the value of a policy may be written $E_{s_0 \sim D}[V^\pi(s_0)] = w \cdot \mu(\pi)$. Given that the reward R is expressible as a linear combination of the features ϕ , the feature expectations for a given policy π completely determine the expected sum of discounted rewards for acting according to that policy.

Let Π denote the set of stationary policies for an MDP. Given two policies $\pi_1, \pi_2 \in \Pi$, we can construct a new policy π_3 by mixing them together. Specifically, imagine that π_3 operates by flipping a coin with bias λ , and with probability λ picks and always acts according to π_1 , and with probability $1 - \lambda$ always acts according to π_2 . From linearity of expectation, clearly we have that $\mu(\pi_3) = \lambda\mu(\pi_1) + (1 - \lambda)\mu(\pi_2)$. Note that the randomization step selecting between π_1 and π_2 occurs only once at the start of a trajectory, and *not* on every step taken in the MDP. More generally, if we have found some set of policies $\pi_1, \dots, \pi_d$, and want to find a new policy whose feature expectations vector is a convex combination $\sum_{i=1}^d \lambda_i \mu(\pi_i)$ ($\lambda_i \geq 0, \sum_i \lambda_i = 1$) of these policies’, then we can do so by mixing together the policies $\pi_1, \dots, \pi_d$, where the probability of picking π_i is given by λ_i .

We assume access to demonstrations by some expert π_E . Specifically, we assume the ability to observe trajectories (state sequences) generated by the expert starting from $s_0 \sim D$ and taking actions according to π_E . It may be helpful to think of the π_E as the optimal policy under the reward function $R^* = w^{*T} \phi$, though we do not require this to hold.

For our algorithm, we will require an estimate of the expert’s feature expectations $\mu_E = \mu(\pi_E)$. Specifi-

cally, given a set of m trajectories $\{s_0^{(i)}, s_1^{(i)}, \dots\}_{i=1}^m$ generated by the expert, we denote the empirical estimate for μ_E by⁴

$$\hat{\mu}_E = \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{\infty} \gamma^t \phi(s_t^{(i)}). \quad (5)$$

In the sequel, we also assume access to a reinforcement learning (RL) algorithm that can be used to solve an MDP\(\mathcal{R}\) augmented with a reward function $R = w^T \phi$. For simplicity of exposition, we will assume that the RL algorithm returns the optimal policy. The generalization to approximate RL algorithms offers no special difficulties; see the full paper. (Abbeel & Ng, 2004)

3. Algorithm

The problem is the following: Given an MDP\(\mathcal{R}\), a feature mapping ϕ and the expert's feature expectations μ_E , find a policy whose performance is close to that of the expert's, on the *unknown* reward function $R^* = w^{*T} \phi$. To accomplish this, we will find a policy $\tilde{\pi}$ such that $\|\mu(\tilde{\pi}) - \mu_E\|_2 \leq \epsilon$. For such a $\tilde{\pi}$, we would have that for any $w \in \mathbb{R}^k$ ($\|w\|_1 \leq 1$),

$$|E[\sum_{t=0}^{\infty} \gamma^t R(s_t) | \pi_E] - E[\sum_{t=0}^{\infty} \gamma^t R(s_t) | \tilde{\pi}]| \quad (6)$$

$$= |w^T \mu(\tilde{\pi}) - w^T \mu_E| \quad (7)$$

$$\leq \|w\|_2 \|\mu(\tilde{\pi}) - \mu_E\|_2 \quad (8)$$

$$\leq 1 \cdot \epsilon = \epsilon \quad (9)$$

The first inequality follows from the fact that $|x^T y| \leq \|x\|_2 \|y\|_2$, and the second from $\|w\|_2 \leq \|w\|_1 \leq 1$. So the problem is reduced to finding a policy $\tilde{\pi}$ that induces feature expectations $\mu(\tilde{\pi})$ close to μ_E . Our apprenticeship learning algorithm for finding such a policy $\tilde{\pi}$ is as follows:

1. Randomly pick some policy $\pi^{(0)}$, compute (or approximate via Monte Carlo) $\mu^{(0)} = \mu(\pi^{(0)})$, and set $i = 1$.
2. Compute $t^{(i)} = \max_{w: \|w\|_2 \leq 1} \min_{j \in \{0, \dots, (i-1)\}} w^T (\mu_E - \mu^{(j)})$, and let $w^{(i)}$ be the value of w that attains this maximum.
3. If $t^{(i)} \leq \epsilon$, then terminate.
4. Using the RL algorithm, compute the optimal policy $\pi^{(i)}$ for the MDP using rewards $R = (w^{(i)})^T \phi$.
5. Compute (or estimate) $\mu^{(i)} = \mu(\pi^{(i)})$.
6. Set $i = i + 1$, and go back to step 2.

Upon termination, the algorithm returns $\{\pi^{(i)} : i = 0 \dots n\}$.

Let us examine the algorithm in detail. On iteration i , we have already found some policies $\pi^{(0)}, \dots, \pi^{(i-1)}$. The optimization in step 2 can be viewed as an inverse reinforcement learning step in which we try to guess

⁴In practice we truncate the trajectories after a finite number H of steps. If $H = H_\epsilon = \log_{\gamma}(\epsilon(1-\gamma))$ is the ϵ -horizon time, then this introduces at most ϵ error into the approximation.

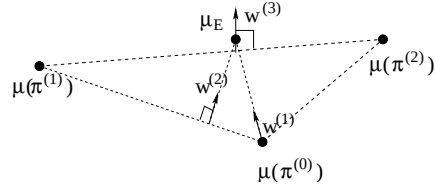


Figure 1. Three iterations for max-margin algorithm.

the reward function being optimized by the expert. The maximization in that step is equivalently written

$$\max_{t, w} \quad t \quad (10)$$

$$\text{s.t.} \quad w^T \mu_E \geq w^T \mu^{(j)} + t, \quad j = 0, \dots, i-1 \quad (11)$$

$$\|w\|_2 \leq 1 \quad (12)$$

From Eq. (11), we see the algorithm is trying to find a reward function $R = w^{(i)} \cdot \phi$ such that $E_{s_0 \sim D}[V^{\pi_E}(s_0)] \geq E_{s_0 \sim D}[V^{\pi^{(i)}}(s_0)] + t$. I.e., a reward on which the expert does better, by a “margin” of t , than any of the i policies we had found previously. This step is similar to one used in (Ng & Russell, 2000), but unlike the algorithms given there, because of the 2-norm constraint on w it cannot be posed as a linear program (LP), but only as a quadratic program.⁵

Readers familiar with support vector machines (SVMs) will also recognize this optimization as being equivalent to finding the maximum margin hyperplane separating two sets of points. (Vapnik, 1998) The equivalence is obtained by associating a label 1 with the expert's feature expectations μ_E , and a label -1 with the feature expectations $\{\mu(\pi^{(j)}) : j = 0 \dots (i-1)\}$. The vector $w^{(i)}$ we want is the unit vector orthogonal to the maximum margin separating hyperplane. So, an SVM solver can also be used to find $w^{(i)}$. (The SVM problem is a quadratic programming problem (QP), so we can also use any generic QP solver.)

In Figure 1 we show an example of what the first three iterations of the algorithm could look like geometrically. Shown are several example $\mu(\pi^{(i)})$, and the $w^{(i)}$'s given by different iterations of the algorithm.

Now, suppose the algorithm terminates, with $t^{(n+1)} \leq \epsilon$. (Whether the algorithm terminates is discussed in Section 4.) Then directly from Eq. (10-12) we have:

$$\forall w \text{ with } \|w\|_2 \leq 1 \exists i \text{ s.t. } w^T \mu^{(i)} \geq w^T \mu_E - \epsilon. \quad (13)$$

Since $\|w^*\|_2 \leq \|w^*\|_1 \leq 1$, this means that there is at least one policy from the set returned by the algorithm, whose performance under R^* is at least as good as the expert's performance minus ϵ . Thus, at this stage, we can ask the agent designer to manually test/examine the policies found by the algorithm, and

⁵Although we previously assumed that the w^* specifying the “true” rewards satisfy $\|w^*\|_1 \leq 1$ (and our theoretical results will use this assumption), we still implement the algorithm using $\|w\|_2 \leq 1$, as in Eq. (12).

pick one with acceptable performance. A slight extension of this method ensures that the agent designer has to examine at most $k + 1$, rather than all $n + 1$, different policies (see footnote 6).

If we do not wish to ask for human help to select a policy, alternatively we can find the point closest to μ_E in the convex closure of $\mu^{(0)}, \dots, \mu^{(n)}$ by solving the following QP:

$$\min \|\mu_E - \mu\|_2, \text{s.t. } \mu = \sum_i \lambda_i \mu^{(i)}, \lambda_i \geq 0, \sum_i \lambda_i = 1.$$

Because μ_E is “separated” from the points $\mu^{(i)}$ by a margin of at most ϵ , we know that for the solution μ we have $\|\mu_E - \mu\|_2 \leq \epsilon$. Further, by “mixing” together the policies $\pi^{(i)}$ according to the mixture weights λ_i as discussed previously, we obtain a policy whose feature expectations are given by μ . Following our previous discussion (Eq. 6-9), this policy attains performance near that of the expert’s on the unknown reward function.⁶

Note that although we called one step of our algorithm an inverse RL step, our algorithm does not necessarily recover the underlying reward function correctly. The performance guarantees of our algorithm only depend on (approximately) matching the feature expectations, not on recovering the true underlying reward function.

3.1. A simpler algorithm

The algorithm described above requires access to a QP (or SVM) solver. It is also possible to change the algorithm so that no QP solver is needed. We will call the previous, QP-based, algorithm the **max-margin** method, and the new algorithm the **projection** method. Briefly, the projection method replaces step 2 of the algorithm with the following:

- Set $\bar{\mu}^{(i-1)} = \bar{\mu}^{(i-2)} + \frac{(\mu^{(i-1)} - \bar{\mu}^{(i-2)})^T (\mu_E - \bar{\mu}^{(i-2)})}{(\mu^{(i-1)} - \bar{\mu}^{(i-2)})^T (\mu^{(i-1)} - \bar{\mu}^{(i-2)})} (\mu^{(i-1)} - \bar{\mu}^{(i-2)})$
(This computes the orthogonal projection of μ_E onto the line through $\bar{\mu}^{(i-2)}$ and $\mu^{(i-1)}$.)
- Set $w^{(i)} = \mu_E - \bar{\mu}^{(i-1)}$
- Set $t^{(i)} = \|\mu_E - \bar{\mu}^{(i-1)}\|_2$

In the first iteration, we also set $w^{(1)} = \mu_E - \mu^{(0)}$ and $\bar{\mu}^{(0)} = \mu^{(0)}$. The full justification for this method is deferred to the full paper (Abbeel and Ng, 2004), but in Sections 4 and 5 we will also give convergence results for it, and empirically compare it to the max-margin

⁶In k -dimensional space, any point that is a convex combination of a set of N points, with $N > k + 1$, can be written as a convex combination of a subset of only $k + 1$ points of the original N points (Caratheodory’s Theorem, Rockafeller, 1970). Applying this to $\mu = \arg \min_{\mu \in Co\{\mu(\pi^{(i)})\}_{i=0}^n} \|\mu E - \mu\|_2$ and $\{\mu(\pi^{(i)})\}_{i=0}^n$ we obtain a set of $k + 1$ policies which is equally close to the expert’s feature expectations and thus have same performance guarantees. (Co denotes convex hull.)

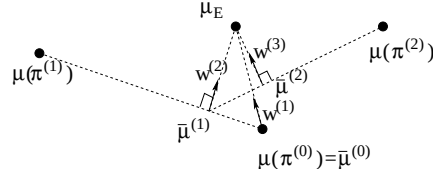


Figure 2. Three iterations for projection algorithm.

algorithm. An example showing three iterations of the projection method is shown in Figure 2.

4. Theoretical results

Most of the results in the previous section were predicated on the assumption that the algorithm terminates with $t \leq \epsilon$. If the algorithm sometimes does not terminate, or if it sometimes takes a very (perhaps exponentially) large number of iterations to terminate, then it would not be useful. The following shows that this is not the case.

Theorem 1. *Let an MDP $\mathcal{R}$, features $\phi : S \mapsto [0, 1]^k$, and any $\epsilon > 0$ be given. Then the apprenticeship learning algorithm (both max-margin and projection versions) will terminate with $t^{(i)} \leq \epsilon$ after at most*

$$n = O\left(\frac{k}{(1-\gamma)^2\epsilon^2} \log \frac{k}{(1-\gamma)\epsilon}\right) \quad (14)$$

iterations.

The previous result (and all of Section 3) had assumed that μ_E was exactly known or calculated. In practice, it has to be estimated from Monte Carlo samples (Eq. 5). We can thus ask about the sample complexity of this algorithm; i.e., how many trajectories m we must observe of the expert before we can guarantee we will approach its performance.

Theorem 2. *Let an MDP $\mathcal{R}$, features $\phi : S \mapsto [0, 1]^k$, and any $\epsilon > 0, \delta > 0$ be given. Suppose the apprenticeship learning algorithm (either max-margin or projection version) is run using an estimate $\hat{\mu}_E$ for μ_E obtained by m Monte Carlo samples. In order to ensure that with probability at least $1 - \delta$ the algorithm terminates after at most a number of iterations n given by Eq. (14), and outputs a policy $\tilde{\pi}$ so that for any true reward $R^*(s) = w^{*T} \phi(s)$ ($\|w^*\|_1 \leq 1$) we have*

$$E[\sum_{t=0}^{\infty} \gamma^t R^*(s_t) | \tilde{\pi}] \geq E[\sum_{t=0}^{\infty} \gamma^t R^*(s_t) | \pi_E] - \epsilon, \quad (15)$$

it suffices that

$$m \geq \frac{2k}{(\epsilon(1-\gamma))^2} \log \frac{2k}{\delta}.$$

The proofs of these theorems are in Appendix A.

In the case where the true reward function R^* does not lie exactly in the span of the basis functions ϕ , the algorithm still enjoys a graceful degradation of performance. Specifically, if $R^*(s) = w^* \cdot \phi(s) + \varepsilon(s)$ for

some residual (error) term $\varepsilon(s)$, then our algorithm will have performance that is worse than the expert’s by no more than $O(\|\varepsilon\|_\infty)$.

5. Experiments

5.1. Gridworld

In our first set of experiments, we used 128 by 128 gridworlds with multiple/sparse rewards. The reward is not known to the algorithm, but we can sample trajectories from an expert’s (optimal) policy. The agent has four actions to try to move in each of the four compass directions, but with 30% chance an action fails and results in a random move. The grid is divided into non-overlapping regions of 16 by 16 cells; we call these 16x16 regions “macrocells.” A small number of the resulting 64 macrocells have positive rewards. For each value of $i = 1, \dots, 64$, there is one feature $\phi_i(s)$ indicating whether that state s is in macrocell i . Thus, the rewards may be written $R^* = (w^*)^T \phi$. The weights w^* are generated randomly so as to give sparse rewards, which leads to fairly interesting/rich optimal policies.⁷

In the basic version, the algorithm is run using the 64-dimensional features ϕ . We also tried a version in which the algorithm knows exactly which macrocells have non-zero rewards (but not their values), so that the dimension of ϕ is reduced to contain only features corresponding to non-zero rewards.

In Figure 3, we compare the max-margin and projection versions of the algorithm, when μ_E is known exactly. We plot the margin $t^{(i)}$ (distance to expert’s policy) vs. the number of iterations, using all 64 macrocells as features. The expert’s policy is the optimal policy with respect to the given MDP. The two algorithms exhibited fairly similar rates of convergence, with the projection version doing slightly better.

The second set of experiments illustrates the performance of the algorithm as we vary the number m of sampled expert trajectories used to estimate μ_E . The performance measure is the value of the best policy in the set output by the algorithm. We ran the algorithm once using all 64 features, and once using only the features that truly correspond to non-zero rewards.⁸ We also report on the performance of three

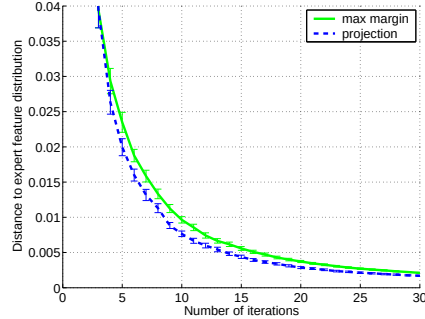


Figure 3. A comparison of the convergence speeds of the max-margin and projection versions of the algorithm on a 128x128 grid. Euclidean distance to the expert’s feature expectations is plotted as a function of the number of iterations. We rescaled the feature expectations by $(1 - \gamma)^k$ such that they are in $[0, 1]^k$. The plot shows averages over 40 runs, with 1 s.e. errorbars.

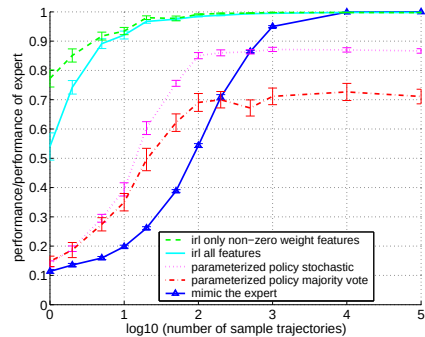


Figure 4. Plot of performance vs. number of sampled trajectories from the expert. (Shown in color, where available.) Averages over 20 instances are plotted, with 1 s.e. errorbars. Note the base-10 logarithm scale on the x -axis.

other simple algorithms. The “mimic the expert” algorithm picks whatever action the expert had taken if it finds itself in a state in which it had previously observed the expert, and picks an action randomly otherwise. The “parameterized policy stochastic” uses a stochastic policy, with the probability of each action constant over each macrocell and set to the empirical frequency observed for the expert in the macrocell. The “parameterized policy majority vote” algorithm takes deterministically the most frequently observed action in the macrocell. Results are shown in Figure 4. Using our algorithm, only a few sampled expert trajectories—far fewer than for the other methods—are needed to attain performance approaching that of the expert. (Note log scale on x -axis.)⁹ Thus, by

mainly in the question of how many times an expert must demonstrate a task before we learn to perform the same task. In particular, we do not rely on the expert’s demonstrations to learn the state transition probabilities.

⁹The parameterized policies never reach the expert’s performance, because their policy class is not rich enough. Their restricted policy class is what makes them do better

⁷Details: We used $\gamma = 0.99$, so the expected horizon is of the order of the gridsize. The true reward function was generated as follows: For each macrocell i ($i = 1, \dots, 64$), with probability 0.9 the reward there is zero ($w_i^* = 0$), and with probability 0.1 a weight w_i^* is sampled uniformly from $[0, 1]$. Finally, w^* is renormalized so that $\|w^*\|_1 = 1$. Instances with fewer than two non-zero entries in w^* are non-interesting and were discarded. The initial state distribution is uniform over all states.

⁸Note that, as in the text, our apprenticeship learning algorithm assumes the ability to call a reinforcement learning subroutine (in this case, an exact MDP solver using value iteration). In these experiments, we are interested

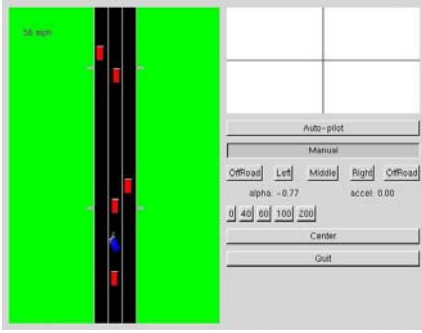


Figure 5. Screenshot of driving simulator.

learning a compact representation of the reward function, our algorithm significantly outperforms the other methods. We also observe that when the algorithm is told in advance which features have non-zero weight in the true reward function, it is able to learn using fewer expert trajectories.

5.2. Car driving simulation

For our second experiment, we implemented a car-driving simulation, and applied apprenticeship learning to try to learn different “driving styles.” A screenshot of our simulator is shown in Figure 5. We are driving on a highway at 25m/s (56mph), which is faster than all the other cars. The MDP has five different actions, three of which cause the car to steer smoothly to one of the lanes, and two of which cause us to drive off (but parallel to) the road, on either the left or the right side. Because our speed is fixed, if we want to avoid hitting other cars it is sometimes necessary to drive off the road.

The simulation runs at 10Hz, and in the experiments that follow, the expert’s features were estimated from a single trajectory of 1200 samples (corresponding to 2 minutes of driving time). There were features indicating what lane the car is currently in (including offroad-left and offroad-right, for a total of five features), and the distance of the closest car in the current lane.¹⁰ Note that a distance of 0 from the nearest car implies a collision. When running the apprenticeship learning algorithm, the step in which reinforcement learning was required was implemented by solving a discretized version of the problem. In all of our experiments, the algorithm was run for 30 iterations, and a policy was selected by inspection (per the discussion in Section 3).

We wanted to demonstrate a variety of different driving styles (some corresponding to highly unsafe driving) to see if the algorithm can mimic the same “style” in every instance. We considered five styles:

1. Nice: The highest priority is to avoid collisions

than the “mimic the expert” algorithm initially.

¹⁰More precisely, we used the distance to the single closest car in its current lane, discretized to the nearest car length between -7 to +2, for a total of 10 features.

with other cars; we also prefer the right lane over the middle lane over the left lane, over driving off-road.

2. Nasty: Hit as many other cars as possible.
3. Right lane nice: Drive in the right lane, but go off-road to avoid hitting cars in the right lane.
4. Right lane nasty: Drive off-road on the right, but get back onto the road to hit cars in the right lane.
5. Middle lane: Drive in the middle lane, ignoring all other cars (thus crashing into all other cars in the middle lane).

After each style was demonstrated to the algorithm (by one of the authors driving in the simulator for 2 minutes), apprenticeship learning was used to try to find a policy that mimics demonstrated style. Videos of the demonstrations and of the resulting learned policies are available at

<http://www.cs.stanford.edu/~pabbeel/irl/>

In every instance, the algorithm was qualitatively able to mimic the demonstrated driving style. Since no “true” reward was ever specified or used in the experiments, we cannot report on the results of the algorithm according to R^* . However, Table 1 shows, for each of the five driving styles, the feature expectations of the expert (as estimated from the 2 minute demonstration), and the feature expectations of the learned controller for the more interesting features. Also shown are the weights w used to generate the policy shown. While our theory makes no guarantee about any set of weights w found, we note that the values there generally make intuitive sense. For instance, in the first driving style, we see negative rewards for collisions and for driving offroad, and larger positive rewards for driving in the right lane than for the other lanes.

6. Discussion and Conclusions

We assumed access to demonstrations by an expert that is trying to maximize a reward function expressible as a linear combination of known features, and presented an algorithm for apprenticeship learning. Our method is based on inverse reinforcement learning, terminates in a small number of iterations, and guarantees that the policy found will have performance comparable to or better than that of the expert, on the expert’s unknown reward function.

Our algorithm assumed the reward function is expressible as a linear function of known features. If the set of features is sufficiently rich, this assumption is fairly unrestrictive. (In the extreme case where there is a separate reward feature for each state-action pair, fully general reward functions can be learned.) However, it remains an important problem to develop methods for learning reward functions that may be non-linear functions of the features, and to incorporate automatic fea-

Table 1. Feature expectations of teacher $\hat{\mu}_E$ and of selected/learned policy $\mu(\tilde{\pi})$ (as estimated by Monte Carlo). and weights w corresponding to the reward function that had been used to generate the policy shown. (Note for compactness, only 6 of the more interesting features, out of a total of 15 features, are shown here.)

		Collision	Offroad Left	LeftLane	MiddleLane	RightLane	Offroad Right
1	$\hat{\mu}_E$	0.0000	0.0000	0.1325	0.2033	0.5983	0.0658
	$\mu(\tilde{\pi})$	0.0001	0.0004	0.0904	0.2287	0.6041	0.0764
	$\tilde{w}$	-0.0767	-0.0439	0.0077	0.0078	0.0318	-0.0035
2	$\hat{\mu}_E$	0.1167	0.0000	0.0633	0.4667	0.4700	0.0000
	$\mu(\tilde{\pi})$	0.1332	0.0000	0.1045	0.3196	0.5759	0.0000
	$\tilde{w}$	0.2340	-0.1098	0.0092	0.0487	0.0576	-0.0056
3	$\hat{\mu}_E$	0.0000	0.0000	0.0000	0.0033	0.7058	0.2908
	$\mu(\tilde{\pi})$	0.0000	0.0000	0.0000	0.0000	0.7447	0.2554
	$\tilde{w}$	-0.1056	-0.0051	-0.0573	-0.0386	0.0929	0.0081
4	$\hat{\mu}_E$	0.0600	0.0000	0.0000	0.0033	0.2908	0.7058
	$\mu(\tilde{\pi})$	0.0569	0.0000	0.0000	0.0000	0.2666	0.7334
	$\tilde{w}$	0.1079	-0.0001	-0.0487	-0.0666	0.0590	0.0564
5	$\hat{\mu}_E$	0.0600	0.0000	0.0000	1.0000	0.0000	0.0000
	$\mu(\tilde{\pi})$	0.0542	0.0000	0.0000	1.0000	0.0000	0.0000
	$\tilde{w}$	0.0094	-0.0108	-0.2765	0.8126	-0.5099	-0.0154

ture construction and feature selection ideas into our algorithms.

It might also be possible to derive an alternative apprenticeship learning algorithm using the dual to the LP that is used to solve Bellman’s equations. (Manne, 1960) Specifically, in this LP the variables are the state/action visitation rates, and it is possible to place constraints on the learned policy’s stationary distribution directly. While there are few algorithms for approximating this dual (as opposed to primal) LP for large MDPs and exact solutions would be feasible only for small MDPs, we consider this an interesting direction for future work.

Acknowledgements. This work was supported by the Department of the Interior/DARPA under contract number NBCHD030010.

References

- Abbeel, P., & Ng, A. Y. (2004). Apprenticeship learning via inverse reinforcement learning. (Full paper.) <http://www.cs.stanford.edu/~pabbeel/irl/>.
- Amit, R., & Mataric, M. (2002). Learning movement sequences from demonstration. *Proc. ICDL*.
- Atkeson, C., & Schaal, S. (1997). Robot learning from demonstration. *Proc. ICML*.
- Demiris, J., & Hayes, G. (1994). A robot controller using learning by imitation.
- Hogan, N. (1984). An organizing principle for a class of voluntary movements. *J. of Neuroscience*, 4, 2745–2754.
- Kuniyoshi, Y., Inaba, M., & Inoue, H. (1994). Learning by watching: Extracting reusable task knowledge from visual observation of human performance. *T-RA*, 10, 799–822.
- Manne, A. (1960). Linear programming and sequential decisions. *Management Science*, 6.
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. *Proc. ICML*.
- Ng, A. Y., & Russell, S. (2000). Algorithms for inverse reinforcement learning. *Proc. ICML*.
- Pomerleau, D. (1989). Alvin: An autonomous land vehicle in a neural network. *NIPS 1*.

Rockafellar, R. (1970). *Convex analysis*. Princeton University Press.

Sammut, C., Hurst, S., Kedzier, D., & Michie, D. (1992). Learning to fly. *Proc. ICML*.

Uno, Y., Kawato, M., & Suzuki, R. (1989). Formation and control of optimal trajectory in human multijoint arm movement. minimum torque-change model. *Biological Cybernetics*, 61, 89–101.

Vapnik, V. N. (1998). *Statistical learning theory*. John Wiley & Sons.

A. Proofs of Theorems

In (Abbeel & Ng, 2004) we give longer, easier to read proofs of the theorems. Due to space constraints, the proofs to follow are quite dense.

In preparation for the proofs, we begin with some definitions. Given a set of policies Π , we define $M = M(\Pi) = \text{Co}\{\mu(\pi) : \pi \in \Pi\}$ to be the convex hull of the set of feature expectations attained by policies $\pi \in \Pi$. Hence, given any vector of feature expectations $\tilde{\mu} \in M$, there is a set of policies $\pi_1, \dots, \pi_n \in \Pi$ and mixture weights $\{\lambda_i\}_{i=1}^n$ ($\lambda_i \geq 0, \sum_{i=1}^n \lambda_i = 1$), so that $\tilde{\mu} = \sum_{i=1}^n \lambda_i \mu(\pi_i)$. Thus, given any point $\tilde{\mu} \in M$, by mixing together policies in Π , we can obtain a new policy whose feature expectations are exactly $\tilde{\mu}$. (Here, mixture policies are as defined in Section 2.) We also define $M^{(i)} = \text{Co}\{\mu(\pi^{(j)}) : j = 0, \dots, i\}$ to be the convex hull of the set of feature expectations of policies found after iterations $0, \dots, i$ of our algorithm. Due to space constraints, we give a full proof only for the case of $\hat{\mu}_E \in M$.¹¹ In reading the proofs, Figure 6 may be helpful for conveying geometric intuition.

The following Lemma establishes improvement in a single iteration of the algorithm.

Lemma 3. *Let there be given an MDP $\setminus R$, features $\phi : S \rightarrow [0, 1]^k$, and a set of policies Π , $\tilde{\mu}^{(i)} \in M$. Let $\pi^{(i+1)}$ be the optimal policy for the MDP $\setminus R$ augmented with reward $R(s) = (\hat{\mu}_E - \tilde{\mu}^{(i)}) \cdot \phi(s)$, i.e. $\pi^{(i+1)} = \arg \max_{\pi} (\hat{\mu}_E -$*

¹¹As mentioned previously, $\hat{\mu}_E$ is a noisy estimate of μ_E . Thus, it may not be a valid feature expectation vector for any policy; i.e., we do not necessarily have $\hat{\mu}_E \in M$. In the extended version of the proofs we consider a small ball with radius ρ centered around $\hat{\mu}_E$ and that intersects M , and prove convergence to this ball. That proof is very similar, but has more technicalities and is significantly longer (Abbeel & Ng, 2004).

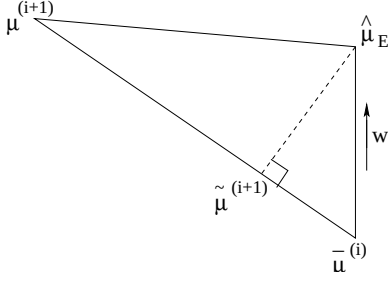


Figure 6. Progress in one iteration step.

$\bar{\mu}^{(i)} \cdot \mu(\pi)$, and let $\tilde{\mu}^{(i+1)} = \frac{(\hat{\mu}_E - \bar{\mu}^{(i)}) \cdot (\mu^{(i+1)} - \bar{\mu}^{(i)})}{\|\mu^{(i+1)} - \bar{\mu}^{(i)}\|_2^2} (\mu^{(i+1)} - \bar{\mu}^{(i)}) + \bar{\mu}^{(i)}$, i.e. the projection of $\hat{\mu}_E$ onto the line through $\mu^{(i+1)} = \mu(\pi^{(i+1)})$, $\bar{\mu}^{(i)}$. Then

$$\frac{\|\hat{\mu}_E - \tilde{\mu}^{(i+1)}\|_2}{\|\hat{\mu}_E - \bar{\mu}^{(i)}\|_2} \leq \frac{k}{\sqrt{k^2 + (1-\gamma)^2 \|\hat{\mu}_E - \bar{\mu}^{(i)}\|_2^2}} \quad (16)$$

and the point $\tilde{\mu}^{(i+1)}$ is a convex combination of $\bar{\mu}^{(i)}$ and $\mu^{(i+1)}$.

Proof. For simplicity of notation, we let the origin of our coordinate system coincide with $\bar{\mu}^{(i)}$. Then

$$\begin{aligned} & \frac{(\bar{\mu}^{(i+1)} - \hat{\mu}_E) \cdot (\tilde{\mu}^{(i+1)} - \hat{\mu}_E)}{\hat{\mu}_E \cdot \hat{\mu}_E} \\ &= \frac{\mu^{(i+1)} \cdot \mu^{(i+1)} - (\mu^{(i+1)} \cdot \hat{\mu}_E)^2}{\mu^{(i+1)} \cdot \mu^{(i+1)}} \\ &\leq \frac{\mu^{(i+1)} \cdot \mu^{(i+1)} - 2\hat{\mu}_E \cdot \mu^{(i+1)} + \hat{\mu}_E \cdot \hat{\mu}_E}{\mu^{(i+1)} \cdot \mu^{(i+1)}} \\ &= \frac{(\mu^{(i+1)} - \hat{\mu}_E) \cdot (\mu^{(i+1)} - \hat{\mu}_E)}{(\mu^{(i+1)} - \hat{\mu}_E) \cdot (\mu^{(i+1)} - \hat{\mu}_E) + \hat{\mu}_E \cdot \hat{\mu}_E + 2(\mu^{(i+1)} - \hat{\mu}_E) \cdot \hat{\mu}_E} \\ &\leq \frac{(\mu^{(i+1)} - \hat{\mu}_E) \cdot (\mu^{(i+1)} - \hat{\mu}_E)}{(\mu^{(i+1)} - \hat{\mu}_E) \cdot (\mu^{(i+1)} - \hat{\mu}_E) + \hat{\mu}_E \cdot \hat{\mu}_E} \\ &\leq \frac{k^2/(1-\gamma)^2}{k^2/(1-\gamma)^2 + \hat{\mu}_E \cdot \hat{\mu}_E}, \end{aligned}$$

where we used in order: the definition of $\tilde{\mu}^{(i+1)}$ and rewriting; $(\mu^{(i+1)} \cdot \hat{\mu}_E - \hat{\mu}_E \cdot \hat{\mu}_E)^2 \geq 0$; rewriting terms; $\mu^{(i+1)} \cdot \hat{\mu}_E \geq \hat{\mu}_E \cdot \hat{\mu}_E$ (since $\pi^{(i+1)} = \arg \max_{\pi} (\hat{\mu}_E - \bar{\mu}^{(i)}) \cdot \mu(\pi)$); all points considered lie in $M \subseteq [0, \frac{1}{1-\gamma}]^k$, so their norms are bounded by $k/(1-\gamma)$. Since the origin coincides with $\bar{\mu}^{(i)}$, this proves Eq. (16). It is easily verified from $\tilde{\mu}^{(i+1)}$'s definition that $\tilde{\mu}^{(i+1)} = \lambda \mu^{(i+1)} + (1-\lambda) \bar{\mu}^{(i)}$, for $\lambda = \frac{\hat{\mu}_E \cdot \mu^{(i+1)}}{\mu^{(i+1)} \cdot \mu^{(i+1)}}$. Also, $\lambda \geq 0$ since $\mu^{(i+1)} \cdot \hat{\mu}_E \geq \hat{\mu}_E \cdot \hat{\mu}_E \geq 0$; and, $\lambda \leq 1$ since $(\mu^{(i+1)} \cdot \hat{\mu}_E)^2 \leq (\mu^{(i+1)} \cdot \mu^{(i+1)}) (\hat{\mu}_E \cdot \hat{\mu}_E) \leq (\mu^{(i+1)} \cdot \mu^{(i+1)}) (\mu^{(i+1)} \cdot \hat{\mu}_E)$ (Cauchy-Schwarz, $\mu^{(i+1)} \cdot \mu^{(i+1)} = \max_{\mu} \hat{\mu}_E \cdot \mu$), which implies $\mu^{(i+1)} \cdot \hat{\mu}_E \leq \mu^{(i+1)} \cdot \mu^{(i+1)}$. $\square$

Proof of Theorem 1. Note that given a point $\bar{\mu}^{(i)}$, Lemma 3 gives a way to construct a point $\tilde{\mu}^{(i+1)} \in M^{(i+1)}$ with a distance to $\hat{\mu}_E$ that is smaller by a factor given by Eq. (16). As long as for the current iterate $\bar{\mu}^{(i)}$ we have $\|\hat{\mu}_E - \bar{\mu}^{(i)}\|_2 \geq \epsilon$, we have $\frac{\|\hat{\mu}_E - \tilde{\mu}^{(i+1)}\|_2}{\|\hat{\mu}_E - \bar{\mu}^{(i)}\|_2} \leq \frac{k}{\sqrt{k^2 + (1-\gamma)^2 \epsilon^2}}$. The max-margin algorithm sets $\bar{\mu}^{(i+1)} = \arg \min_{\mu \in M^{(i+1)}} \|\hat{\mu}_E - \mu\|_2$. So we have $t^{(i+1)}/t^{(i)} \leq \frac{k}{\sqrt{k^2 + (1-\gamma)^2 \epsilon^2}}$. The projection algorithm sets $\bar{\mu}^{(i+1)} =$

$\tilde{\mu}^{(i+1)}$, so it keeps track of a single point (which is a convex combination of previously obtained points), for which the distance $t^{(\cdot)}$ is reduced in each iteration by the same factor: $t^{(i+1)}/t^{(i)} \leq \frac{k}{\sqrt{k^2 + (1-\gamma)^2 \epsilon^2}}$. Since the maximum distance in M is $\sqrt{k}/(1-\gamma)$, we have

$$t^{(i)} \leq \left(\frac{\sqrt{k}}{\sqrt{k^2 + (1-\gamma)^2 \epsilon^2}} \right)^i \frac{\sqrt{k}}{1-\gamma}. \quad (17)$$

So we have $t^{(i)} \leq \epsilon$ if

$$\begin{aligned} i &\geq \log \frac{\sqrt{k}}{(1-\gamma)\epsilon} / \log \frac{\sqrt{(1-\gamma)^2 \epsilon^2 + k}}{\sqrt{k}} \\ &= O\left(\frac{k}{(1-\gamma)^2 \epsilon^2} \log \frac{k}{(1-\gamma)\epsilon}\right). \end{aligned}$$

This completes the proof. $\square$

Proof of Theorem 2. Recall $\phi \in [0, 1]^k$ so $\mu \in [0, \frac{1}{1-\gamma}]^k$. Let μ_i denote the i 'th component of μ . Then applying the Hoeffding inequality on the m -sample estimate $(1-\gamma)\hat{\mu}_i$ of $(1-\gamma)\mu_i \in [0, 1]$ gives

$$P((1-\gamma)|\mu_i - \hat{\mu}_i| > \tau) \leq 2\exp(-2\tau^2 m). \quad (18)$$

Using Eq. (18) for all i and the union bound gives

$$P(\exists i \in \{1 \dots k\}. (1-\gamma)|\mu_i - \hat{\mu}_i| > \tau) \leq 2k\exp(-2\tau^2 m),$$

which can be rewritten as

$$P((1-\gamma)\|\mu_E - \hat{\mu}_E\|_\infty \leq \tau) \geq 1 - 2k\exp(-2\tau^2 m).$$

Substituting $\tau = (1-\gamma)\epsilon/(2\sqrt{k})$ gives

$$P(\|\mu_E - \hat{\mu}_E\|_\infty \leq \frac{\epsilon}{2\sqrt{k}}) \geq 1 - 2k\exp(-2(\frac{\epsilon(1-\gamma)}{2\sqrt{k}})^2 m),$$

where k is the dimension of the feature vectors ϕ and feature expectations μ , and m is the number of sample trajectories used for the estimate $\hat{\mu}_E$. So if we take $m \geq \frac{2k}{(\epsilon(1-\gamma))^2} \log \frac{2k}{\delta}$ then with probability at least $(1-\delta)$ we have that $\|\mu_E - \hat{\mu}_E\|_\infty \leq \frac{\epsilon}{2\sqrt{k}}$. Using $\|\cdot\|_2 \leq \sqrt{k}\|\cdot\|_\infty$ for k -dimensional spaces, we get

$$\|\mu_E - \hat{\mu}_E\|_2 \leq \frac{\epsilon}{2} \text{ w.p. } 1-\delta \text{ for } m \geq \frac{2k}{(\epsilon(1-\gamma))^2} \log \frac{2k}{\delta}. \quad (19)$$

Theorem 1 guarantees that after a sufficient number of iterations we have $t^{(i)} \leq \frac{\epsilon}{2}$. Thus, assuming that m is as given in Eq. (19), we have that with probability $1-\delta$ (and after sufficient iterations) the algorithm will return a policy $\tilde{\pi}$ such that $\tilde{\mu} = \mu(\tilde{\pi})$ satisfies

$$\begin{aligned} \|\tilde{\mu} - \mu_E\|_2 &\leq \|\tilde{\mu} - \hat{\mu}_E\|_2 + \|\hat{\mu}_E - \mu_E\|_2 \\ &\leq t^{(i)} + \frac{\epsilon}{2} \\ &\leq \epsilon. \end{aligned} \quad (20)$$

Using this (and keeping in mind that $\|\cdot\|_2 \leq \|\cdot\|_1$ and so $\|w^*\|_1 \leq 1$ implies $\|w^*\|_2 \leq 1$) we can prove that w.p. $1-\delta$,

$$\begin{aligned} &|E[\sum_{t=0}^{\infty} \gamma^t R^*(s^{(t)})|\tilde{\pi}] - E[\sum_{t=0}^{\infty} \gamma^t R^*(s^{(t)})|\pi_E]| \\ &= |(w^*)^T(\tilde{\mu} - \mu_E)| \\ &\leq \epsilon, \end{aligned}$$

where we used in order the definitions of w, μ ; Cauchy-Schwarz; and Eq. (20). $\square$